Table of contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Overview of the Project	1
1.2 Motivation and Contribution	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Existing System	3
2.2 Proposed System	4
2.3 Feasibility Study	5
3. SYSTEM REQUIREMENTS	6
3.1 Hardware Requirements	6
3.2 Functional Requirements	6
3.3 Non-Functional Requirements	7
4. SYSTEM DESIGN	8
4.1 System Architecture	8
4.2 Database Design	9
4.3 Incident Collection Structure	9
4.4 ER Diagram	10
4.5 Data Flow Overview	10

5. SYSTEM IMPLEMENTATION 11
5.1 Frontend Implementation 11
5.2 Backend Implementation 12
5.3 Database Implementation 13

6. TESTING 14

7. CONCLUSION 15

8. FUTURE ENHANCEMENTS 16

9. REFERENCES 17

APPENDIX 18
Appendix A: Screenshots 18
Appendix B: Sample Data 19
Appendix C: Tools Used 19
Appendix D: Features 20

ABSTRACT

The Disaster Management System is a web-based application developed to manage disaster-related incidents efficiently. The system allows users to report incidents such as floods, fires, or accidents, and enables administrators to monitor, track, and resolve these incidents. The Disaster Management System is a web-based application developed to manage disaster-related incidents efficiently. The system allows users to report incidents such as floods, fires, or accidents, and enables administrators to monitor, track, and resolve these incidents. This system provides two types of login roles: **User and Admin**. Users can add incidents and track their status, while admins can view all incidents, analyze data through dashboards, and mark incidents as resolved. The application improves response time, ensures better communication, and provides a centralized platform for disaster management.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Login Screen of Disaster Management System	11
Figure 2	User Dashboard Screen	11
Figure 3	Admin Dashboard Screen	12
Figure 4	Add Incident Form	12
Figure 5	Incident List Display	13

LIST OF TABLES

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Incident Collection Structure	9
Table 2	System Requirements	6

ACRONYMS

- **DMS** : Disaster Management System
- **JDBC** : Java Database Connectivity
- **GUI** : Graphical User Interface
- **SQL** : Structured Query Language

1. INTRODUCTION

1.1 Overview of the Project

Disaster management is a critical process that requires timely reporting and efficient handling of incidents. Traditional methods of reporting disasters are often manual, slow, and inefficient. The Disaster Management System is designed to provide a digital solution where users can easily report incidents and administrators can manage them effectively. The system ensures proper tracking of incidents from reporting to resolution.

1.2 Motivation and Contribution of Project

The motivation behind this project is to overcome the limitations of manual disaster reporting systems, such as:

- Delayed communication
- Lack of centralized data
- Poor tracking of incidents

This project introduces a modern web-based solution to address these issues.

1.3 Objectives of the Project

The main objectives of this project are:

- To develop a user-friendly web application
- To allow users to report disaster incidents
- To provide an admin dashboard for monitoring incidents
- To track incident status (Pending / Resolved)
- To improve response efficiency

1.4 Scope of the Project

The system can be used in:

- Government disaster management departments
- Colleges and institutions
- Organizations and communities

Future scope includes integration with real-time systems and mobile applications.

2. SYSTEM ANALYSIS

2.1 Existing System

In the current scenario, disaster management is often handled using traditional and manual methods such as phone calls, written reports, and physical registers. These methods are inefficient and lack proper coordination between users and authorities.

When a disaster occurs, individuals report incidents through informal communication channels, which may lead to delays in response. There is no centralized system to store or manage data, making it difficult to track the status of incidents

Some of the major drawbacks of the existing system are:

- **Lack of real-time updates:** Information is not updated instantly, causing delays in response.
- **Data redundancy and loss:** Manual records can be misplaced or damaged.
- **Inefficient communication:** No proper channel between reporting users and authorities.
- **No tracking system:** It is difficult to monitor whether an issue is resolved or still pending.
- **Time-consuming process:** Manual handling of incidents increases response time.

Due to these limitations, there is a need for a digital system that can efficiently manage disaster-related information.

2.2 Proposed System

The proposed Disaster Management System is a web-based application designed to overcome the limitations of the existing system. It provides a centralized platform where users can report incidents and administrators can manage them efficiently.

The system introduces a structured approach with role-based access:

User Role:

- Can report disaster incidents
- Can view the status of reported incidents

Admin Role:

- Can view all incidents
- Can monitor dashboard statistics
- Can update incident status from pending to resolved

The system ensures that all data is stored in a database and can be accessed anytime. It provides real-time updates and improves coordination between users and administrators.

Key features of the proposed system include:

- Online incident reporting
- Dashboard with summary statistics
- Status tracking (Pending/Resolved)
- Role-based login system
- User-friendly interface

Completed till here……

2.3 Feasibility Study

The feasibility study evaluates whether the proposed system is practical and beneficial.

1. Technical Feasibility

The system is developed using modern technologies such as React.js, Node.js, and MongoDB, which are widely used and easy to implement. The required tools and resources are readily available.

2. Economic Feasibility

The system is cost-effective as it uses open-source technologies. No expensive software or hardware is required, making it affordable.

3. Operational Feasibility

The system is user-friendly and easy to operate. Both users and administrators can interact with the system without requiring technical expertise.

4 Advantages of Proposed System

The proposed system offers several advantages over the existing system:

- **Improved efficiency:** Faster reporting and processing of incidents
- **Centralized data management:** All data is stored in a single database
- **Real-time updates:** Immediate reflection of changes
- **User-friendly interface:** Easy navigation and operation
- **Better decision making:** Admin dashboard provides insights

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The system does not require high-end hardware. Minimum requirements include:

- Processor: Intel Core i3 or above
- RAM: 4GB or higher
- Hard Disk: 500GB
- Internet connection

3.3 Functional Requirements

The system must provide the following functionalities:

- User login and authentication
- Add new incidents
- View all incidents
- Update incident status
- Dashboard display for admin

3.4 Non-Functional Requirements

- **Performance:** System should respond quickly
- **Security:** Only authorized users can access
- **Usability:** Easy to use interface
- **Reliability:** System should work without failures

4. SYSTEM DESIGN

4.1 System Architecture

The Disaster Management System follows a **three-tier architecture**, consisting of:

1. **Frontend** **(Presentation_Layer)**

   Built using React.js, it provides the user interface for interaction.

2. **Backend** **(Application_Layer)**

   Developed using Node.js and Express.js, it handles business logic and API requests.

3. **Database** **(Data_Layer)**

   MongoDB is used to store all incident-related data.

4.2 DATABASE DESIGN

The database is designed using MongoDB, which is a NoSQL database. It stores data in the form of documents (JSON format).

In this system, a single collection named **"Incidents"** is used to store all disaster-related information.

4.3 Incident Collection Structure

Each record in the database represents one incident. The structure of the Incident collection is as follows:

Field Name	Data Type	Description
_id	ObjectId	Unique identifier (auto-generated)
type	String	Type of disaster (Flood, Fire, etc.)
description	String	Details about the incident
location	String	Location of the incident
status	String	Status (Pending / Resolved)
createdAt	Date	Date and time of creation

4.4 ER Diagram (Explanation)

The system uses a simple structure with one main entity:

Entity: Incident

Attributes:

- Type

- Description
- Location
- Status
- CreatedAt

Since the system does not use multiple tables or relationships, the design is simple and efficient.

4.5 Data Flow Overview

1. User submits incident through frontend
2. Backend receives request
3. Data is stored in MongoDB
4. Admin retrieves and updates status
5. Updated data is shown to users

6. SYSTEM IMPLEMENTATION

5.1 Frontend Implementation

The frontend is developed using React.js, which provides a dynamic and responsive user interface.

Features implemented:

- Login page with role selection
- Forms for adding incidents
- Dashboard for admin
- Incident display cards
- CSS styling for better UI

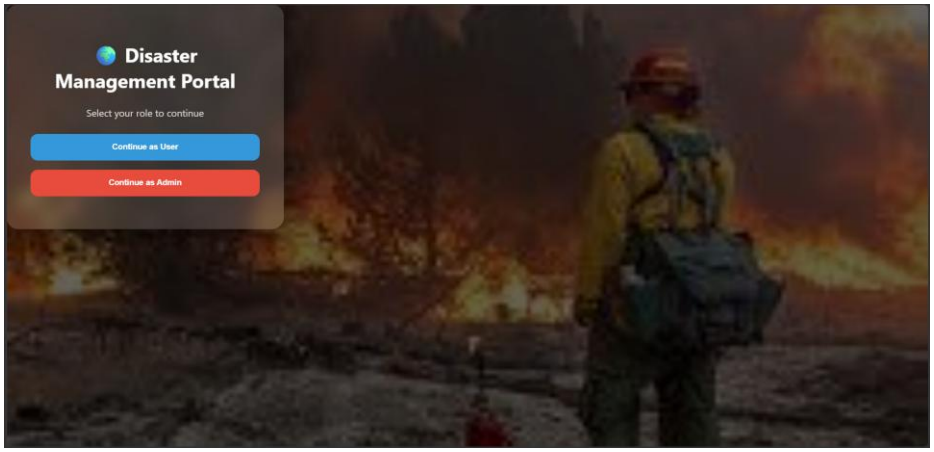

Figure 1: Login Screen of Disaster Management System

User Screen:

Provides options such as Add and View Disasters.

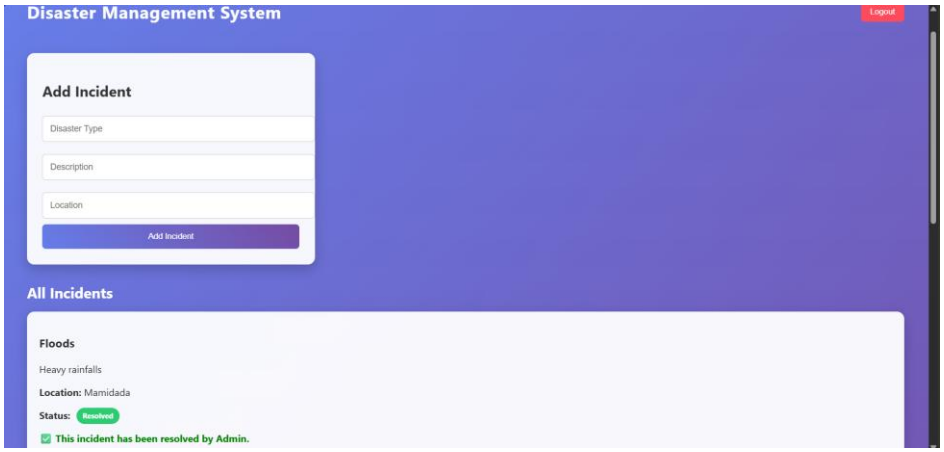

Figure 2: Main Menu Screen of Employee Management System

Admin Screen:

Access to view, update, delete the disasters.

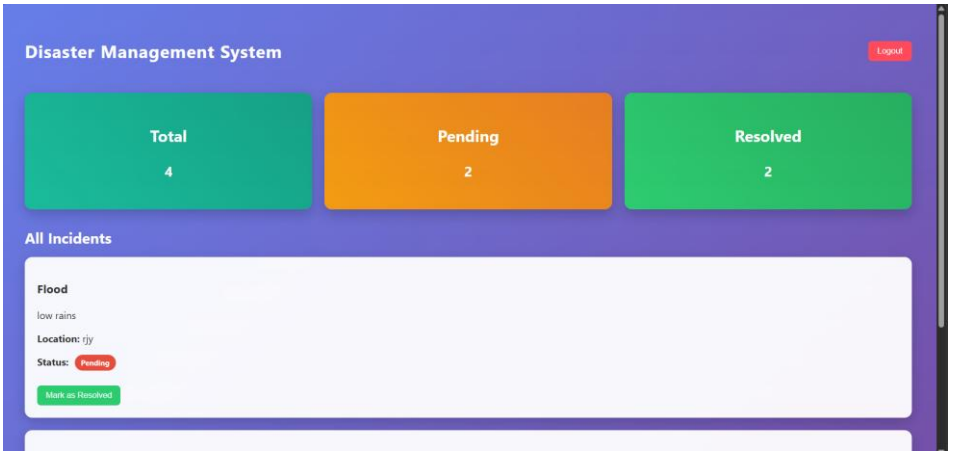

Figure 3: Screen of Admin Disaster Management System

5.2 Backend Implementation

The backend is developed using Node.js and Express.js.

Features implemented:

- REST API for CRUD operations
- Handling HTTP requests (GET, POST, PUT, DELETE)
- Middleware integration
- Database connectivity

5.3 Database Implementation

MongoDB is used as the database.

Schema Structure:

- Type: String
- Description: String
- Location: String
- Status: String (Pending/Resolved)
- CreatedAt: Date

6.TESTING

The system is tested using different test cases:

- Login functionality test
- Add incident test
- Update status test
- Display data test

All functionalities worked correctly without errors.

7. CONCLUSION

The Disaster Management System developed in this project successfully addresses the challenges faced in traditional disaster reporting and management systems. The system provides a centralized and efficient platform for users to report incidents and for administrators to monitor and manage them.

Through the implementation of modern web technologies such as React.js, Node.js, and MongoDB, the system ensures fast performance, real-time updates, and secure data handling. The role-based login system enhances security and ensures that users and administrators have controlled access to functionalities.

The user interface is designed to be simple and interactive, allowing users to easily submit incident details and view their status. The administrator dashboard provides a clear overview of total incidents, pending cases, and resolved cases, which helps in effective decision-making.

One of the key achievements of this project is the ability to track incidents from creation to resolution. This ensures transparency and accountability in disaster management. Additionally, the system reduces manual effort, minimizes errors, and improves response time.

Overall, the project meets all the objectives defined at the beginning. It demonstrates how technology can be effectively used to improve disaster management processes and enhance communication between users and authorities.

8. FUTURE ENHANCEMENTS

The current system works efficiently, but it can be further improved by adding the following features:

1. Secure Authentication

Implement advanced login security using JWT and encrypted passwords to improve data protection.

2. Image Upload

Allow users to upload images along with incident details for better understanding of the situation.

3. Real-Time Notifications

Provide notifications through email or SMS when the status of an incident is updated.

4. GPS Location Integration

Automatically detect user location and display incidents on a map.

5. Mobile Application

Develop a mobile app so users can report incidents easily from their smartphones.

6. Advanced Dashboard

Enhance admin dashboard with charts and graphs for better analysis.

9. REFERENCES

REFERENCES

1. React.js Documentation
   https://reactjs.org
2. Node.js Documentation
   https://nodejs.org
3. Express.js Documentation
   https://expressjs.com
4. MongoDB Documentation
   https://www.mongodb.com/docs
5. Mongoose Documentation
   https://mongoosejs.com
6. W3Schools Web_Development_Tutorials
   https://www.w3schools.com
7. Stack Overflow (Technical_Discussions)
   https://stackoverflow.com

APPENDIX

Appendix A: Screenshots

The following screenshots are included to demonstrate the working of the system:

- Login Page (Role Selection)
- User Login Page
- Admin Dashboard
- Add Incident Form
- Incident List Display
- Status Update (Resolved)

Appendix B: Sample Data

Example of an incident stored in the database:

```
{
  "type": "Fire",
  "description": "Fire accident in residential area",
  "location": "Vijayawada",
  "status": "Pending"
}
```

Appendix C: Tools and Technologies Used

- Frontend: React.js
- Backend: Node.js, Express.js
- Database: MongoDB
- Code Editor: Visual Studio Code
- API Testing Tool: Postman
- Browser: Google Chrome

Appendix D: Key Features of the System

- Role-based login (Admin & User)
- Add new incidents
- View incident list
- Update status (Pending → Resolved)
- Dashboard summary for admin
- Simple and user-friendly interface